

JavaCUP

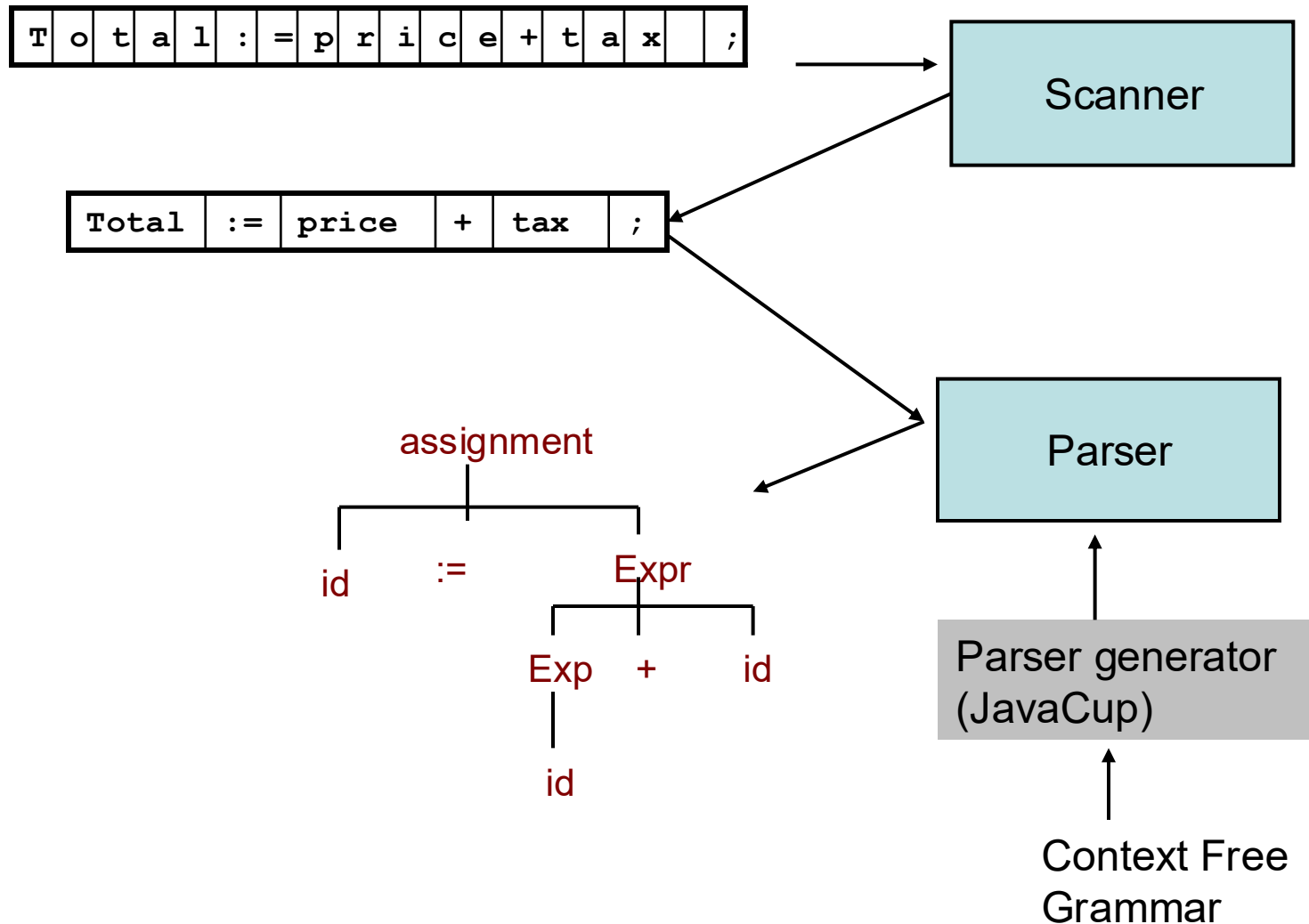
- JavaCUP (Construct Useful Parser) is a parser generator
- Produce a parser written in java, itself is also written in Java;
- There are many parser generators.
 - YACC (Yet Another Compiler-Compiler) for C programming language (dragon book chapter 4.9);
- There are also many parser generators written in Java
 - JavaCC;
 - ANTLR;
 - ...



More on classification of java parser generators

- Bottom-up Parser Generators Tools
 - JavaCUP;
 - SableCC, The Sable Compiler Compiler www.sablecc.org
- Top-down Parser Generators Tools
 - ANTLR, Another Tool for Language Recognition www.antlr.org
 - JavaCC, Java Compiler Compiler www.webgain.com/java_cc

What is a parser generator



Steps to use JavaCup

- Write a javaCup specification (cup file)
 - Defines the grammar and actions in a file (say, calc.cup)
- Run javaCup to generate a parser
 - `java java_cup.Main calc.cup`
 - Notice the package prefix `java_cup` before `Main`;
 - Will generate `parser.java` and `sym.java` (default class names, which can be changed)
- Write your program that uses the parser
 - For example, `UseParser.java`
- Compile and run your program

Example 1: parse an expression and evaluate it

- Grammar for arithmetic expression

$\text{expr} \rightarrow \text{expr} \text{ '+' } \text{expr}$
| $\text{expr} \text{ '-' } \text{expr}$
| $\text{expr} \text{ '*' } \text{expr}$
| $\text{expr} \text{ '/' } \text{expr}$
| '(' expr ')'
| number

- Example

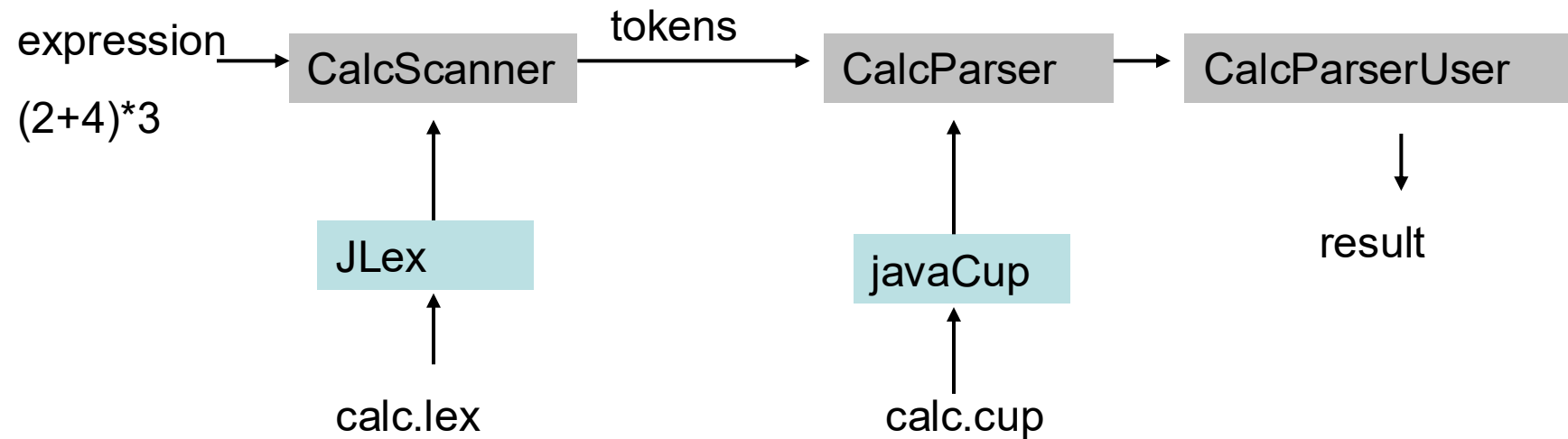
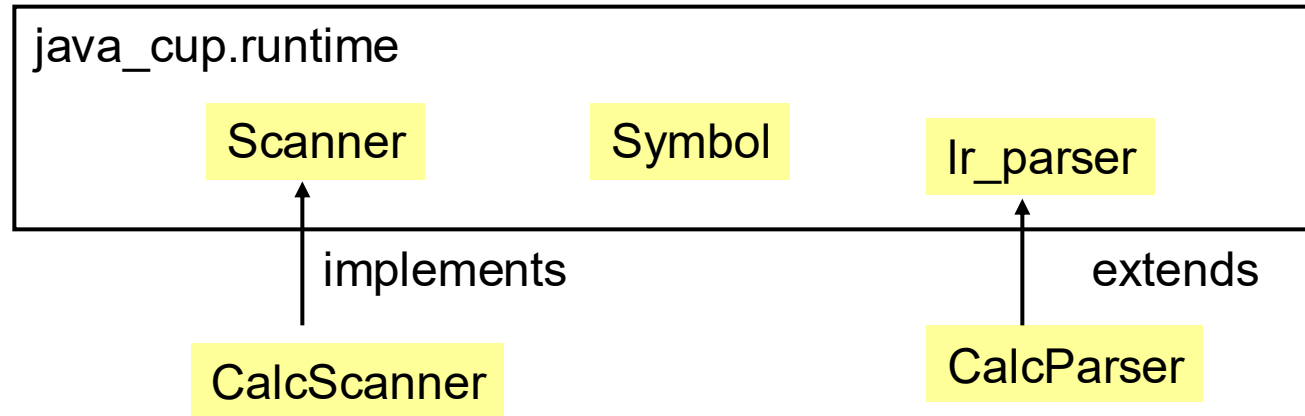
$(2+4)*3$ is an expression

- Our tasks:

- Tell whether an expression like **$(2+4)*3$** is syntactically correct;
- Evaluate the expression (we are actually producing an interpreter for the “expression language”).

The overall picture

```
public interface Scanner {  
    public Symbol next_token() throws  
        java.lang.Exception;  
}
```



Calculator javaCup specification (calc.cup)

```
terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;  
terminal Integer NUMBER;  
non terminal Integer expr;  
precedence left PLUS, MINUS;  
precedence left TIMES, DIVIDE;  
expr ::= expr PLUS expr  
      | expr MINUS expr  
      | expr TIMES expr  
      | expr DIVIDE expr  
      | LPAREN expr RPAREN  
      | NUMBER  
      ;
```

- Is the grammar ambiguous?
- Add precedence and associativity
 - left means, that $a + b + c$ is parsed as $(a + b) + c$
 - lowest precedence comes first, so $a + b * c$ is parsed as $a + (b * c)$
- How can we get PLUS, NUMBER, ...?
 - They are the terminals returned by the scanner.
- How to connect with the scanner?

Ambiguous grammar error

- If we enter the grammar as below:

Expression ::= Expression PLUS Expression;

- Without precedence JavaCUP will tell us:

Shift/Reduce conflict found in state #4

between Expression ::= Expression PLUS Expression ()
and Expression ::= Expression () PLUS Expression
under symbol PLUS

Resolved in favor of shifting.

- The grammar is ambiguous!
- Telling JavaCUP that PLUS is left associative helps.

Corresponding scanner specification (calc.lex)

```
1.import java_cup.runtime.Symbol;
2.import java_cup.runtime.Scanner;
3.%%
4.%implements java_cup.runtime.Scanner
5.%type Symbol
6.%function next_token
7.%class CalcScanner
8.%eofval{ return null;
9.%eofval}
10.NUMBER = [0-9]+
11.%%
12."+" { return new Symbol(CalcSymbol.PLUS); }
13."-" { return new Symbol(CalcSymbol.MINUS); }
14."*" { return new Symbol(CalcSymbol.TIMES); }
15."/" { return new Symbol(CalcSymbol.DIVIDE); }
16.{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext()));}
17.\r|\n|. {}
```

- Connection with the parser
 - imports java_cup.runtime.*, Symbol, Scanner.
 - implements Scanner
 - next_token: defined in Scanner interface
 - CalcSymbol, PLUS, MINUS, ...
 - new Integer(yytext())

Run JLex

D:\214>java JLex.Main calc.lex

- note the package prefix JLex
- program text generated: calc.lex.java

D:\214>javac calc.lex.java

- classes generated: CalcScanner.class

Generated CalcScanner class

```
1. import java_cup.runtime.Symbol;
2. Import java_cup.runtime.Scanner;
3. class CalcScanner implements java_cup.runtime.Scanner {
4.     ...
5.     public Symbol next_token () {
6.         ...
7.         case 3: { return new Symbol(CalcSymbol.MINUS); }
8.         case 6: { return new Symbol(CalcSymbol.NUMBER, new
           Integer(yytext())); }
9.         ...
10.    }
11. }
```

- Interface Scanner is defined in java_cup.runtime package

```
public interface Scanner {
    public Symbol next_token() throws
        java.lang.Exception;
}
```

Run javaCup

- Run javaCup to generate the parser
 - D:\214>java java_cup.Main -parser CalcParser -symbols CalcSymbol calc.cup
 - classes generated:
 - CalcParser;
 - CalcSymbol;
- Compile the parser and relevant classes
 - D:\214>javac CalcParser.java CalcSymbol.java CalcParserUser.java
- Use the parser
 - D:\214>java CalcParserUser

The token class Symbol.java

```
1. public class Symbol {
2.     public int sym, left, right;
3.     public Object value;
4.     public Symbol(int id, int l, int r, Object o) {
5.         this(id); left = l; right = r; value = o;
6.     }
7.     public Symbol(int id, Object o) { this(id, -1, -1, o); }
8.     public Symbol(int sym_num) { .. }
9.     public String toString() { return "#" + sym; }
10. }
```

- **Instance variables:**

- sym: the symbol type;
- left: left position in the original input file;
- right: right position in the original input file;
- value: the lexical value.

- **Recall the action in lex file:**

```
[0-9]+ {return new Symbol(CalcSymbol.NUMBER, new Integer(yytext()));}
"+" { return new Symbol(CalcSymbol.PLUS); }
```

CalcSymbol.java (default name is sym.java)

```
1. public class CalcSymbol {  
2.     public static final int MINUS = 3;  
3.     public static final int DIVIDE = 5;  
4.     public static final int NUMBER = 8;  
5.     public static final int EOF = 0;  
6.     public static final int PLUS = 2;  
7.     public static final int error = 1;  
8.     public static final int RPAREN = 7;  
9.     public static final int TIMES = 4;  
10.    public static final int LPAREN = 6;  
11. }
```

- Contain token declaration, one for each token (terminal); Generated from the terminal list in cup file
 - terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
 - terminal Integer NUMBER
- Used by scanner to refer to symbol types, e.g.,
 - return new Symbol(CalcSymbol.PLUS);
- Class name comes from –symbols directive.

```
java java_cup.Main -parser CalcParser -symbols CalcSymbol calc.cup
```

The program that uses the CalcParser

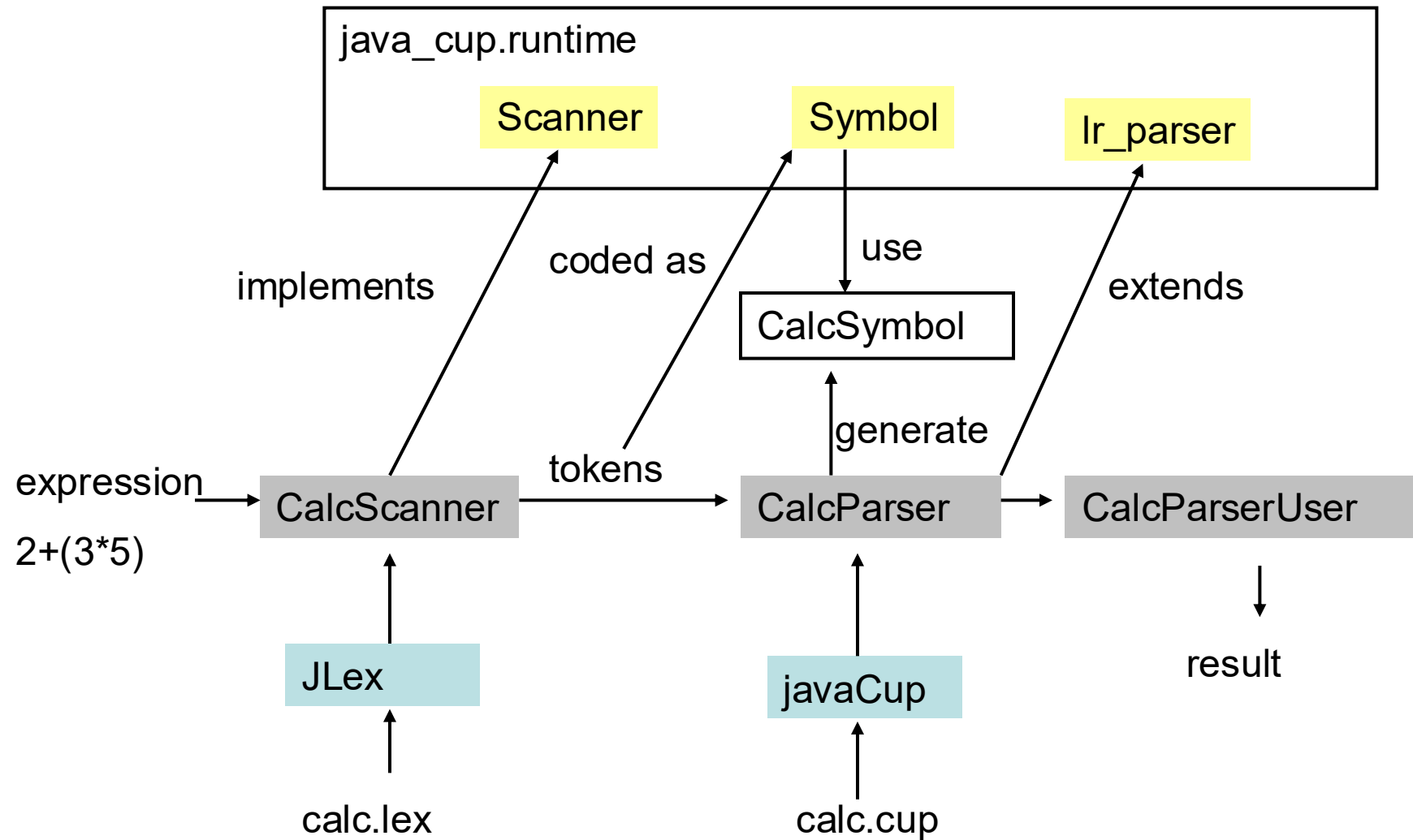
```
1. import java.io.*;
2. class CalcParserUser {
3.     public static void main(String[] args) throws IOException{
4.         File inputFile = new File ("d:/214/calc.input");
5.         CalcParser parser= new CalcParser
6.             (new CalcScanner(new FileInputStream(inputFile)));
7.         parser.parse();
8.     }
9. }
```

- The input text to be parsed can be any input stream (in this example it is a `FileInputStream`);
- The first step is to construct a parser object. A parser can be constructed using a scanner.
 - this is how scanner and parser get connected.
- If there is no error report, the expression in the input file is correct.

Recap

- To write a parser, how many things you need to write?
 - cup file;
 - lex file;
 - a program to use the parser;
- To run a parser, how many things you need to do?
 - Run javaCup, to generate the parser;
 - Run JLex, to generate the scanner;
 - Compile the scanner, the parser, the relevant classes, and the class using the parser;
 - relevant classes: CalcSymbol, Symbol
 - Run the class that uses the parser.

Recap (cont.)



Evaluate the expression

- The previous specification only indicates the success or failure of a parser. No semantic action is associated with grammar rules.
- To calculate the expression, we must add java code in the grammar to carry out actions at various points.
- Form of the semantic action:

```
expr:e1 PLUS expr:e2  
{: RESULT=new Integer(e1+ e2) ;  
:}
```

- Actions (java code) are enclosed within a pair {: :}. Note that it is different from JLex action code bracket
- Labels e1, e2: the objects that represent the corresponding terminal or non-terminal;
- RESULT: The type of RESULT should be the same as the type of the corresponding non-terminals. e.g., expr is of type Integer, so RESULT is of type integer.
- In the cup file, you need to specify expr is of Integer type.

```
non terminal Integer expr;
```

Change the calc.cup

```
1.  terminal PLUS, MINUS, TIMES, DIVIDE, LPAREN, RPAREN;
2.  terminal Integer NUMBER;
3.  non terminal Integer expr;
4.  precedence left PLUS, MINUS;
5.  precedence left TIMES, DIVIDE;
6.  expr ::= expr:e1 PLUS expr:e2 {:
7.      RESULT = e1 + e2; :}
8.  | expr:e1 MINUS expr:e2 {:
9.      RESULT = e1 - e2; :}
10. | expr:e1 TIMES expr:e2 {:
11.    RESULT = e1 * e2; :}
12. | expr:e1 DIVIDE expr:e2 {:
13.    RESULT = e1 / e2; :}
14. | LPAREN expr:e RPAREN {: RESULT = e; :}
15. | NUMBER:e {: RESULT = e; :}
```

- How do you guarantee NUMBER is of Integer type?
- Yytext() returns a String

```
{NUMBER} { return new Symbol(CalcSymbol.NUMBER, new Integer(yytext())); }
```

Change CalcParserUser

```
1. import java.io.*;
2. class CalcParserUser {
3.     public static void main(String[] a) throws Exception{
4.         CalcParser parser= new CalcParser(
5.             new CalcScanner(new FileReader("calc.input")));
6.         Integer result= (Integer)parser.parse().value;
7.         System.out.println("result is "+ result);
8.     }
9. }
```

- Why the result of `parser().value` can be casted into an `Integer`? Can we cast that into other types?
 - This is determined by the type of `expr`, which is the head of the first production in javaCup specification:

`non terminal Integer expr;`

Calc: second round

- Calc program syntax

program → statement | statement program

statement → assignment SEMI

assignment → ID EQUAL expr

expr → expr PLUS expr

 | expr MULTI expr

 | LPAREN expr RPAREN

 | NUMBER

 | ID

- Example program:

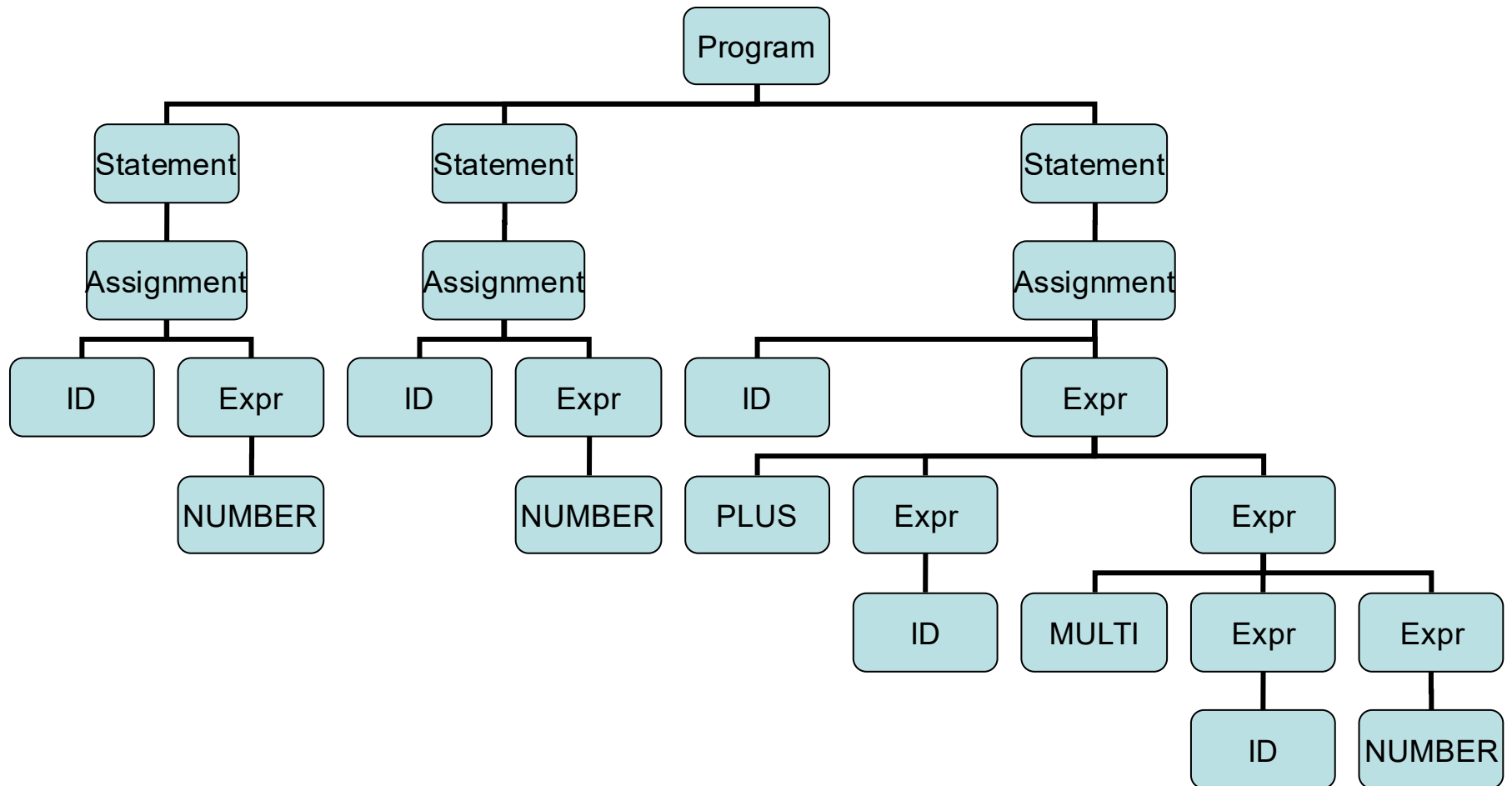
- X=1; y=2; z=x+y*2;

- Task: generate and display the parse tree in XML

```
- <program>
- <Assignment>
  <lhs>x</lhs>
  - <Expr>
    <value>1</value>
  </Expr>
</Assignment>
- <Assignment>
  <lhs>y</lhs>
  - <Expr>
    <value>2</value>
  </Expr>
</Assignment>
- <Assignment>
  <lhs>z</lhs>
  - <Expr>
    <op>+</op>
    <Expr>x</Expr>
  - <Expr>
    <op>*</op>
    <Expr>y</Expr>
  - <Expr>
    <value>2</value>
  </Expr>
</Expr>
</Expr>
</Assignment>
</program>
```

Abstract syntax tree

$X=1; y=2; z=x+y*2;$



OO Design Rationale

- Write a class for every non-terminal
 - Program, Statement, Assignment, Expr
- Write an abstract class for non-terminal which has alternatives
 - Given a rule: `statement` $\rightarrow$ `assignment` | `ifStatement`
 - Statement should be an abstract class;
 - Assignment should extends Statement;
- Semantic part of the CUP file will construct the object;
 - `assignment ::= ID:e1 EQUAL expr:e2`
`{: RESULT = new Assignment(e1, e2); :}`
- The first rule will return the top level object (the Program object)
 - the result of parsing is a Program object
- It is similar to XML DOM parser.

Calc2.cup

```
1.terminal String ID, LPAREN, RPAREN, EQUAL, SEMI, PLUS, MULTI;
2.terminal Integer  NUMBER;
3.non terminal Expr expr;
4.non terminal Statement statement;
5.non terminal Program program;
6.non terminal Assignment assignment;
7.precedence left PLUS;
8.precedence left MULTI;
9.program ::= statement:e { : RESULT = new Program(e) ; :}
10.    | statement:e1 program:e2 { : RESULT=new Program(e1, e2) ; :} ;
11.statement ::= assignment:e SEMI { : RESULT = e ; :} ;
12.assignment ::= ID:e1 EQUAL expr:e2
13.    { : RESULT = new Assignment(e1, e2) ; :} ;
14.expr ::= expr:e1 PLUS:e expr:e2 { : RESULT=new Expr(e1,e2,e) ; :}
15.    | expr:e1 MULTI:e expr:e2 { : RESULT=new Expr(e1,e2,e) ; :}
16.    | LPAREN expr:e RPAREN { : RESULT = e ; :}
17.    | NUMBER:e { : RESULT= new Expr(e) ; :}
18.    | ID:e { : RESULT = new Expr(e) ; :}
19.    ;
```

- Common bugs in assignments: ; { : :}

Program class

```
1. import java.util.*;
2. public class Program {
3.     private Vector statements;
4.     public Program(Statement s) {
5.         statements = new Vector();
6.         statements.add(s);
7.     }
8.     public Program(Statement s, Program p) {
9.         statements = p.getStatements();
10.        statements.add(s);
11.    }
12.    public Vector getStatements(){ return statements; }
13.    public String toXML() { ... ... }
14. }
```

```
Program ::= statement:e      {: RESULT=new Program(e); :}
| statement:e1 program:e2   {: RESULT=new Program(e1, e2); :}
```

Assignment statement class

```
1.class Assignment extends Statement{
2.    private String lhs;
3.    private Expr rhs;
4.    public Assignment(String l, Expr r){
5.        lhs=l;
6.        rhs=r;
7.    }
8.    String toXML(){
9.        String result("<Assignment>");
10.        result += "<lhs>" + lhs + "</lhs>";
11.        result += rhs.toXML();
12.        result += "</Assignment>";
13.        return result;
14.    }
15.}
```

```
assignment ::= ID : e1 EQUAL expr : e2
           { : RESULT = new Assignment(e1, e2) ; : }
```

Expr class

```
1. public class Expr {  
2.     private int value;  
3.     private String id;  
4.     private Expr left;  
5.     private Expr right;  
6.     private String op;  
7.     public Expr(Expr l, Expr r, String o){ left=l; right=r; op=o; }  
8.     public Expr(Integer i){ value=i;}  
9.     public Expr(String i){ id=i;}  
10.    public String toXML() { ... }  
11.}
```

```
expr ::= expr:e1 PLUS:e expr:e2  
      { : RESULT = new Expr(e1, e2, e); :}  
| expr:e1 MULTI:e expr:e2 { : RESULT = new Expr(e1, e2, e); :}  
| LPAREN expr:e RPAREN { : RESULT = e; :}  
| NUMBER:e { : RESULT= new Expr(e); :}  
| ID:e      { : RESULT = new Expr(e); :}
```

Calc2.lex

```
1. import java_cup.runtime.*;
2. %%
3. %implements java_cup.runtime.Scanner
4. %type Symbol
5. %function next_token
6. %class Calc2Scanner
7. %eofval{ return null;
8. %eofval}
9. IDENTIFIER = [a-zA-Z][a-zA-Z0-9_]*
10. NUMBER = [0-9]+
11. %%
12. "+" { return new Symbol(Calc2Symbol.PLUS, yytext()); }
13. "*" { return new Symbol(Calc2Symbol.MULTI, yytext()); }
14. "=" { return new Symbol(Calc2Symbol.EQUAL, yytext()); }
15. ";" { return new Symbol(Calc2Symbol.SEMI, yytext()); }
16. "(" { return new Symbol(Calc2Symbol.LPAREN, yytext()); }
17. ")" { return new Symbol(Calc2Symbol.RPAREN, yytext()); }
18. {IDENTIFIER} {return new Symbol(Calc2Symbol.ID, yytext()); }
19. {NUMBER} { return new Symbol(Calc2Symbol.NUMBER, new
    Integer(yytext())); }
20. \n|\r|. { }
```

Calc2Parser User

```
1.class ProgramProcessor {
2.  public static void main(String[] args) throws IOException{
3.    File inputFile = new File ("d:/214/calc2.input");
4.    Calc2Parser parser= new Calc2Parser(
5.        new Calc2Scanner(new FileInputStream(inputFile)));
6.    Program pm= (Program)parser.debug_parse().value;
7.    String xml=pm.toXML();
8.    System.out.println("result is "+ xml);
9.  }
10.}
```

- `Debug_parser()`: print out debug info, such as the current token being processed, the rule being applied.
 - Useful to debug javacup specification.
- Parsing result value is of `Program` type—this is decided by the type of the program rule:

```
Program ::= statement:e { : RESULT = new Program(e); : }
        | statement:e1 program:e2 { : RESULT=new Program(e1, e2); : }
        ;
```

Another way to define the expression syntax

```
terminal PLUS, MINUS, TIMES, DIV, LPAREN, RPAREN;  
terminal NUMLIT;  
non terminal Expression, Term, Factor;  
start with Expression;  
Expression ::= Expression PLUS Term  
             | Expression MINUS Term  
             | Term  
             ;  
Term ::= Term TIMES Factor  
       | Term DIV Factor  
       | Factor  
       ;  
Factor ::= NUMLIT  
        | LPAREN Expression RPAREN  
        ;
```

Debug the grammar

```
import java.io.*;
class A3User {
    public static void main(String[] args) throws Exception {
        File inputFile = new File ("A3.tiny");
        A3Parser parser= new A3Parser(new A3Scanner(new
        FileInputStream(inputFile)));
        Integer result =(Integer)parser.debug_parse().value;
        FileWriter fw=new FileWriter(new File("A3.output"));
        fw.write("Number of methods: "+ result);
        fw.close();
    }
}
```

Parser will print out processed symbols and the current symbol that is causing the problem

Run all the programs using one command

- Save the following into a file:

```
java JLex.Main A3.lex  
java java_cup.Main -parser A3Parser -symbols A3Symbol < A3.cup  
javac A3.lex.java A3Parser.java A3Symbol.java A3User.java  
java A3User
```

- Under unix
 - Can be any file name. say run2140
 - Type: “chmod 755 run2140”
 - Type “run2140”
- Under windows
 - Save as “run2140.bat”
 - Type “run2140”
- It is script programming

More flexible

- Script program (say named run2140)

```
java JLex.Main $1.lex
```

```
mv $1.lex.java $1Scanner.java
```

```
java java_cup.Main -parser $1Parser -symbols $1Symbol A3Lu.cup
```

```
javac $1Scanner.java A3Parser.java A3Symbol.java A3User.java
```

```
java $1User
```

```
more $1.output
```

- Run the scrip program with parameter

```
> run2140 A3
```